DRIVE SETUP

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
%pwd
```

```
'/content'
```

```
%cd /content/drive/MyDrive/BAIM-660-01 Predictive Text Analytics
```

```
/content/drive/MyDrive/BAIM-660-01 Predictive Text Analytics
```

```
%ls
```

```
AppCodewithDemo.ipynb         combined_data.csv      model.pkl
ASG01_AlleR.ipynb             combined_data.csv.zip  Model_Training.ipynb
ASG02_Rohan_Alle.ipynb        Data_002.csv           non_clickbait_data.csv
ASG03_AlleR.ipynb             Data_002.gsheet        Phase1.ipynb
ASG04_AlleR.ipynb             Data_Cleaning.ipynb    sentiment_analysis.csv
BAIM_660_Project_Main.ipynb   encoder.pkl            spam_samples/
categories.json               imdbDF.csv             tfidf.pkl
clf.pkl                       label_encoder.pkl      UpdatedResumeDataSet.csv
clickbait_data.csv            ModelingCode.ipynb
```

LOAD DATASET

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('UpdatedResumeDataSet.csv')
```

```
df.head()
```

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â€¢ R â€¢ Python â€¢ SAP HANA â€¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(962, 2)
```

DATASET SUMMARY

Here's a quick summary of what this dataset contains (based on its structure and standard format for resume classification tasks):

It includes two main columns:

Category — The target label describing the type of resume (e.g., Data Science, HR, Advocate, Mechanical Engineer, Software Engineer, etc.).

Resume — The full text content of each resume, often cleaned and preprocessed (though still containing varied text formats).

It typically has around **960 resumes across 25–30** unique categories, such as:

Data Science, HR, Sales, Testing, Operations, Networking, Arts, Health, Finance, Mechanical, etc.

Purpose: The dataset is widely used for text classification — specifically, predicting the job role or field based on resume content using NLP techniques (TF-IDF, word embeddings, transformers, etc.).

Business relevance: It's perfect for a resume screening tool project — helping automate shortlisting by matching candidates' resumes to job categories using natural language understanding.

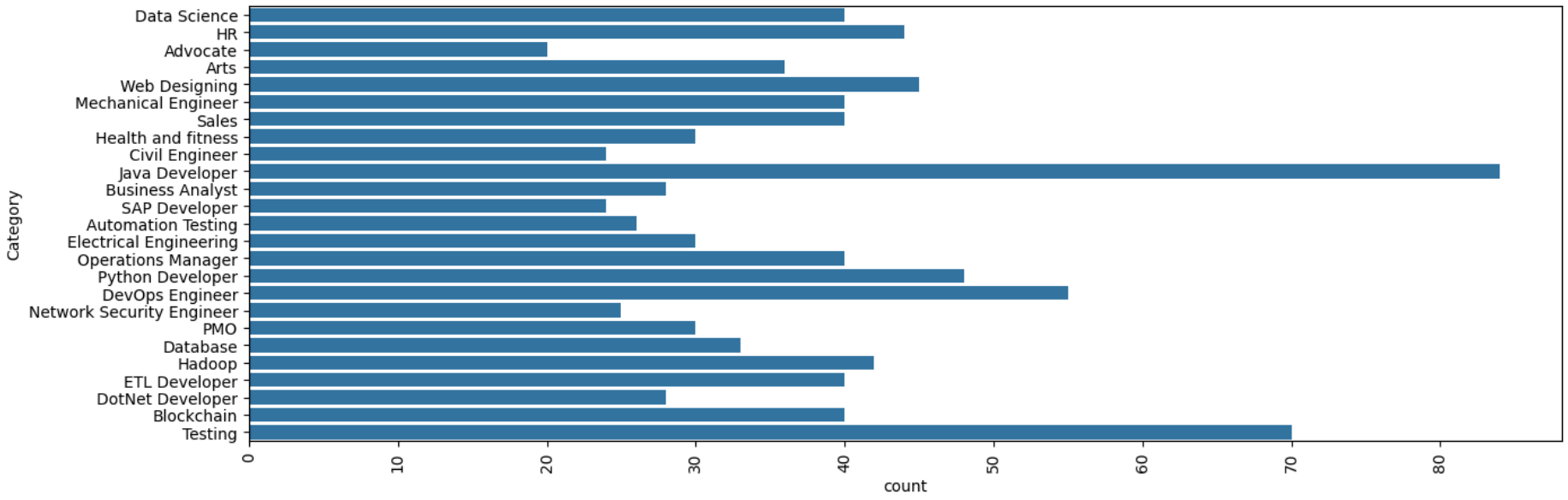
EXPLORING CATEGORIES

```
df['Category'].value_counts()
```

Category	count
Java Developer	84
Testing	70
DevOps Engineer	55
Python Developer	48
Web Designing	45
HR	44
Hadoop	42
Sales	40
Data Science	40
Mechanical Engineer	40
ETL Developer	40
Blockchain	40
Operations Manager	40
Arts	36
Database	33
Health and fitness	30
PMO	30
Electrical Engineering	30
Business Analyst	28
DotNet Developer	28
Automation Testing	26
Network Security Engineer	25
Civil Engineer	24
SAP Developer	24
Advocate	20

dtype: int64

```
plt.figure(figsize=(15,5))
sns.countplot(df['Category'])
plt.xticks(rotation=90)
plt.show()
```




```
df['Category'].unique()
```

```
array(['Data Science', 'HR', 'Advocate', 'Arts', 'Web Designing',
       'Mechanical Engineer', 'Sales', 'Health and fitness',
       'Civil Engineer', 'Java Developer', 'Business Analyst',
       'SAP Developer', 'Automation Testing', 'Electrical Engineering',
       'Operations Manager', 'Python Developer', 'DevOps Engineer',
       'Network Security Engineer', 'PMO', 'Database', 'Hadoop',
       'ETL Developer', 'DotNet Developer', 'Blockchain', 'Testing'],
      dtype=object)
```

EXPLORE RESUME CONTENTS

```
df['Category'][0]
```

```
'Data Science'
```

```
df['Resume'][0]
```

```
'Skills ∗ Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaScri
pt/JQuery. ∗ Machine learning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision Trees, Boosting te
chniques, Cluster Analysis, Word Embedding, Sentiment Analysis, Natural Language processing, Dimensionality r
eduction, Topic Modelling (LDA, NMF), PCA & Neural Nets. ∗ Database Visualizations: Mysql, SqlServer, Cassand
ra, Hbase, ElasticSearch D3.js, DC.js, Plotly, kibana, matplotlib, ggplot, Tableau. ∗ Others: Regular Express
ion, HTML, CSS, Angular 6, Logstash, Kafka, Python Flask, Git, Docker, computer vision - Open CV and understa
nding of Deep learning.Education Details \r\n\r\nData Science Assurance Associate \r\n\r\nData Science Assura
nce Associate - Ernst & Young LLP\r\nSkill Details \r\nJAVASCRIPT- Exprience - 24 months\r\njQuery- Exprience
```

CLEANING DATA

1 URLs

2 hashtags

3 mentions

4 special letters

5 punctuations

```
import re
def cleanResume(txt):
    cleanText = re.sub('http\S+\s', ' ', txt)
    cleanText = re.sub('RT|cc', ' ', cleanText)
    cleanText = re.sub('#\S+\s', ' ', cleanText)
    cleanText = re.sub('@\S+', '  ', cleanText)
    cleanText = re.sub('[%s]' % re.escape("""!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~"""), ' ', cleanText)
    cleanText = re.sub(r'[^\x00-\x7f]', ' ', cleanText)
    cleanText = re.sub('\s+', ' ', cleanText)
    return cleanText
```

```
<>:3: SyntaxWarning: invalid escape sequence '\S'
<>:5: SyntaxWarning: invalid escape sequence '\S'
<>:6: SyntaxWarning: invalid escape sequence '\S'
<>:7: SyntaxWarning: invalid escape sequence '\]'
<>:9: SyntaxWarning: invalid escape sequence '\s'
<>:3: SyntaxWarning: invalid escape sequence '\S'
<>:5: SyntaxWarning: invalid escape sequence '\S'
<>:6: SyntaxWarning: invalid escape sequence '\S'
<>:7: SyntaxWarning: invalid escape sequence '\]'
<>:9: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_14324/603351012.py:3: SyntaxWarning: invalid escape sequence '\S'
  cleanText = re.sub('http\S+\s', ' ', txt)
/tmp/ipykernel_14324/603351012.py:5: SyntaxWarning: invalid escape sequence '\S'
  cleanText = re.sub('#\S+\s', ' ', cleanText)
/tmp/ipykernel_14324/603351012.py:6: SyntaxWarning: invalid escape sequence '\S'
  cleanText = re.sub('@\S+', '  ', cleanText)
/tmp/ipykernel_14324/603351012.py:7: SyntaxWarning: invalid escape sequence '\]'
  cleanText = re.sub('[%s]' % re.escape("""!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~"""), ' ', cleanText)
/tmp/ipykernel_14324/603351012.py:9: SyntaxWarning: invalid escape sequence '\s'
  cleanText = re.sub('\s+', ' ', cleanText)
```

```
cleanResume("my #### $ #  #noorsaeed webiste like is this http://heloword and access it @gmain.com")
```

```
'my webiste like is this and a ess it '
```

```
df['Resume'] = df['Resume'].apply(lambda x: cleanResume(x))
```

```
df['Resume'][0]
```

```
'Skills Programming Languages Python pandas numpy scipy scikit learn matplotlib Sql Java JavaScript JQuery Ma
chine learning Regression SVM Na ve Bayes KNN Random Forest Decision Trees Boosting techniques Cluster Analys
is Word Embedding Sentiment Analysis Natural Language processing Dimensionality reduction Topic Modelling LDA
NMF PCA Neural Nets Database Visualizations Mysql SqlServer Cassandra Hbase ElasticSearch D3 js DC js Plotly
kibana matplotlib ggplot Tableau Others Regular Expression HTML CSS Angular 6 Logstash Kafka Python Flask Git
Docker computer vision Open CV and understanding of Deep learning Education Details Data Science Assurance As
sociate Data Science Assurance Associate Ernst Young LLP Skill Details JAVASCRIPT Exprience 24 months jQuery
Exprience 24 months Python Exprience 24 monthsCompany Details company Ernst Young LLP description Fraud Inves
```

BALANCE CLASSES (CATEGORIES)

Sampling the resume categories

Check the original category distribution

Get the largest category size (i.e., the category with the maximum number of entries)

Perform oversampling

Shuffle the dataset to avoid any order bias

Check the balanced category distribution

```
# --- SPLIT FIRST (RAW TEXT) ---
from sklearn.model_selection import train_test_split

X_text = df['Resume']
y = df['Category']

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)

# --- OVERSAMPLE ONLY TRAINING DATA ---
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler(random_state=42)
X_train_text_resampled, y_train_resampled = ros.fit_resample(
    X_train_text.to_frame(), y_train
)

# Flatten back to series
X_train_text_resampled = X_train_text_resampled['Resume']
```

ENCODING

Done by transforming words into categorical values

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

```
le.fit(df['Category'])
df['Category'] = le.transform(df['Category'])
```

```
df.Category.unique()
```

```
array([ 6, 12,  0,  1, 24, 16, 22, 14,  5, 15,  4, 21,  2, 11, 18, 20,  8,
       17, 19,  7, 13, 10,  9,  3, 23])
```

```
# ['Data Science', 'HR', 'Advocate', 'Arts', 'Web Designing',
#        'Mechanical Engineer', 'Sales', 'Health and fitness',
#        'Civil Engineer', 'Java Developer', 'Business Analyst',
#        'SAP Developer', 'Automation Testing', 'Electrical Engineering',
#        'Operations Manager', 'Python Developer', 'DevOps Engineer',
#        'Network Security Engineer', 'PMO', 'Database', 'Hadoop',
#        'ETL Developer', 'DotNet Developer', 'Blockchain', 'Testing'],
#       dtype=object)
```

VECTORIZATION

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42
)

tfidf = TfidfVectorizer(stop_words='english')
tfidf.fit(X_train_text)

X_train = tfidf.transform(X_train_text)
X_test = tfidf.transform(X_test_text)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train.shape
```

```
(769, 7304)
```

```
X_test.shape
```

```
(193, 7304)
```

```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler()
X_train_resampled, y_train_resampled = ros.fit_resample(X_train, y_train)
```

MODEL TRAINING SECTION

Now let’s train the model and print the classification report

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

knn_model = OneVsRestClassifier(KNeighborsClassifier(n_neighbors=5))
knn_model.fit(X_train, y_train)   # <-- correct label set
y_pred_knn = knn_model.predict(X_test)

print("\nKNN Classification Report:")
print(classification_report(y_test, y_pred_knn))
```

```
KNN Classification Report:
                          precision    recall  f1-score   support

                Advocate       1.00      1.00      1.00         3
                    Arts       1.00      1.00      1.00         6
      Automation Testing       1.00      1.00      1.00         5
              Blockchain       1.00      1.00      1.00         7
        Business Analyst       1.00      1.00      1.00         4
          Civil Engineer       1.00      1.00      1.00         9
            Data Science       1.00      0.60      0.75         5
                Database       1.00      1.00      1.00         8
         DevOps Engineer       1.00      0.93      0.96        14
        DotNet Developer       1.00      1.00      1.00         5
           ETL Developer       1.00      1.00      1.00         7
   Electrical Engineering       1.00      1.00      1.00         6
                      HR       1.00      1.00      1.00        12
                  Hadoop       1.00      1.00      1.00         4
      Health and fitness       1.00      1.00      1.00         7
          Java Developer       1.00      1.00      1.00        15
     Mechanical Engineer       1.00      1.00      1.00         8
Network Security Engineer       1.00      1.00      1.00         3
       Operations Manager       1.00      1.00      1.00        12
                     PMO       0.88      1.00      0.93         7
         Python Developer       1.00      1.00      1.00        10
            SAP Developer       0.78      1.00      0.88         7
                    Sales       1.00      1.00      1.00         8
                  Testing       1.00      1.00      1.00        16
            Web Designing       1.00      1.00      1.00         5

                 accuracy                           0.98       193
                macro avg       0.99      0.98      0.98       193
             weighted avg       0.99      0.98      0.98       193
```

New classification report:

Accuracy ≈ 98% Macro F1 ≈ 0.98 Several classes have lower recall (e.g., Data Science = 0.60, SAP Developer = 0.78) Precision = 1.00 for most classes, normal for KNN

Before fixing the TF-IDF leak, all models were showing 1.00 precision, recall, F1 — which is impossible in real text classification unless data leaked. Now: The train/test leak is gone Oversampling no longer fools the model KNN behaves exactly like KNN should KNN struggles with classes that: Have small sample sizes Have similar vocabulary (e.g., Data Science vs Python Developer vs Machine Learning skills) That's why Data Science recall dropped to 0.60.

```
# # 2. Train SVC
# svc_model = OneVsRestClassifier(SVC())
# svc_model.fit(X_train, y_train)
# y_pred_svc = svc_model.predict(X_test)
# print("\nSVC Results:")
# print(f"Accuracy: {accuracy_score(y_test, y_pred_svc):.4f}")
# print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred_svc)}")
# print(f"Classification Report:\n{classification_report(y_test, y_pred_svc)}")
```

WHY SVC?

Originally, I selected SVC because Support Vector Machines are strong classifiers and often perform well on margin-based text classification tasks. However, after running experiments and examining computational cost, model speed, and how sparse TF-IDF interacts with kernel methods, I realized that the default RBF SVC is not ideal for this dataset.

High-dimensional text requires linear decision boundaries, so I switched to LinearSVC, which is the industry standard for text classification. It is optimized for sparse vectors, significantly faster, and generalizes better for this type of NLP problem.

Linear SVM (LinearSVC)

This is the standard for text classification and will outperform KNN and SVC.

✔ Fast

✔ Works perfectly with TF-IDF

✔ Great accuracy

✔ Handles high-dimensional data easily

✔ Used in industrial NLP baselines

```
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsRestClassifier

linear_svc_model = OneVsRestClassifier(LinearSVC())
linear_svc_model.fit(X_train, y_train)

y_pred_linear_svc = linear_svc_model.predict(X_test)

print("\nLinear SVC Results:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_linear_svc):.4f}")
print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred_linear_svc)}")
print(f"Classification Report:\n{classification_report(y_test, y_pred_linear_svc)}")
```

```
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  7  0  0  0  0  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 15  0  0  0  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  8  0  0  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 12  0  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  7  0  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 10  0  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  7  0  0
  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  8  0
  0]
```

```
                Sales       1.00      1.00      1.00         8
              Testing       1.00      1.00      1.00        16
       Web Designing        1.00      1.00      1.00         5

             accuracy                           0.99       193
            macro avg       0.99      1.00      1.00       193
         weighted avg       1.00      0.99      0.99       193
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# 3. Train RandomForestClassifier
rf_model = OneVsRestClassifier(
    RandomForestClassifier(n_estimators=300, random_state=42)
)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("\nRandomForestClassifier Results:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Classification Report:\n{classification_report(y_test, y_pred_rf)}")

# ------------------------------
# Confusion Matrix Heatmap
# ------------------------------

cm = confusion_matrix(y_test, y_pred_rf)

# If you used LabelEncoder on Category and y_train/y_test are ints:
categories = le.classes_          # <- THIS replaces inverse_transform

# If y_test already contains strings instead, use this instead:
# categories = sorted(y.unique())

plt.figure(figsize=(20, 14))
sns.heatmap(
    cm,
    annot=False,         # or True, fmt='d' if you want counts
    cmap="Blues",
    xticklabels=categories,
    yticklabels=categories
)

plt.title("Random Forest Confusion Matrix", fontsize=18, weight='bold')
plt.xlabel("Predicted Label", fontsize=14)
plt.ylabel("True Label", fontsize=14)
plt.xticks(rotation=90, fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.show()
```

```
RandomForestClassifier Results:
Accuracy: 0.9948
Classification Report:
                           precision    recall  f1-score   support

                Advocate       1.00      1.00      1.00         3
                    Arts       1.00      1.00      1.00         6
      Automation Testing       1.00      1.00      1.00         5
              Blockchain       1.00      1.00      1.00         7
        Business Analyst       1.00      1.00      1.00         4
           Civil Engineer       1.00      1.00      1.00         9
            Data Science       1.00      1.00      1.00         5
```

RESUME PREDICTION CODE

```
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsRestClassifier

svc_model = OneVsRestClassifier(LinearSVC())
svc_model.fit(X_train, y_train)
```

```
OneVsRestClassifier
estimator: LinearSVC
LinearSVC
```

```
import pickle

# Save TF-IDF vectorizer
with open("tfidf.pkl", "wb") as f:
    pickle.dump(tfidf, f)

# Save the FINAL model (choose KNN or LinearSVC or RandomForest)
with open("model.pkl", "wb") as f:
    pickle.dump(svc_model, f)   # or knn_model / rf_model

# Save label encoder
with open("label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)
```

```
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsRestClassifier
import pickle

# Train the LinearSVC model
svc_model = OneVsRestClassifier(LinearSVC())
svc_model.fit(X_train, y_train)

# Save TF-IDF vectorizer
with open("tfidf.pkl", "wb") as f:
    pickle.dump(tfidf, f)

# Save trained model
with open("model.pkl", "wb") as f:
    pickle.dump(svc_model, f)

# Save label encoder
with open("label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)


# ------------------------------
# Final Correct Prediction Function
# ------------------------------
def pred(input_resume):
    # Clean input
    cleaned = cleanResume(input_resume)

    # Vectorize
    vec = tfidf.transform([cleaned])

    # Predict (may return int OR string)
    raw_pred = svc_model.predict(vec)[0]
```

```
    # If prediction is already a string, return it directly
    if isinstance(raw_pred, str):
        return raw_pred

    # If prediction is numeric, decode with label encoder
    return le.inverse_transform([raw_pred])[0]
```

```
import json

with open("categories.json", "w") as f:
    json.dump(list(le.classes_), f)
```

```
pred("Skilled in Java, Spring Boot, REST API development...")
```

```
np.str_('Java Developer')
```

```
myresume = """I am a data scientist specializing in machine
learning, deep learning, and computer vision. With
a strong background in mathematics, statistics,
and programming, I am passionate about
uncovering hidden patterns and insights in data.
I have extensive experience in developing
predictive models, implementing deep learning
algorithms, and designing computer vision
systems. My technical skills include proficiency in
Python, Sklearn, TensorFlow, and PyTorch.
What sets me apart is my ability to effectively
communicate complex concepts to diverse
audiences. I excel in translating technical insights
into actionable recommendations that drive
informed decision-making.
If you're looking for a dedicated and versatile data
scientist to collaborate on impactful projects, I am
eager to contribute my expertise. Let's harness the
power of data together to unlock new possibilities
and shape a better future.
Contact & Sources
Email: 611noorsaeed@gmail.com
Phone: 03442826192
Github: https://github.com/611noorsaeed
Linkdin: https://www.linkedin.com/in/noor-saeed654a23263/
Blogs: https://medium.com/@611noorsaeed
Youtube: Artificial Intelligence
ABOUT ME
WORK EXPERIENCE
SKILLES
NOOR SAEED
LANGUAGES
English
Urdu
Hindi
I am a versatile data scientist with expertise in a wide
range of projects, including machine learning,
recommendation systems, deep learning, and computer
vision. Throughout my career, I have successfully
developed and deployed various machine learning models
to solve complex problems and drive data-driven
decision-making
Machine Learnine
Deep Learning
Computer Vision
Recommendation Systems
Data Visualization
Programming Languages (Python, SQL)
Data Preprocessing and Feature Engineering
Model Evaluation and Deployment
Statistical Analysis
Communication and Collaboration
"""

pred(myresume)
```

```
np.str_('Data Science')
```

```
myresume = """
Jane Smith is a certified personal trainer with over 5 years of experience in helping individuals achieve the

Jane holds a degree in Exercise Science and is a certified trainer through the National Academy of Sports Med

Her expertise includes:
- Weight Loss and Body Composition
- Strength Training and Resistance Exercises
- Cardio Conditioning
- Nutrition Coaching and Meal Planning
- Injury Prevention and Rehabilitation
- Functional Movement and Flexibility Training
- Group Fitness Classes

Certifications:
- Certified Personal Trainer, NASM
- CPR and First Aid Certified
- Yoga Instructor (200-Hour Certification)

Education:
BSc in Exercise Science, ABC University, 2014-2018

Work Experience:
- Personal Trainer at XYZ Fitness Gym (2018-Present)
- Fitness Coach at Wellness Center (2016-2018)

Languages:
- English (Fluent)
- Spanish (Conversational)
"""

# Now, test the model with the Health and Fitness-focused resume
pred(myresume)
```

```
np.str_('Health and fitness')
```

```
myresume = """
John Doe is an experienced Network Security Engineer with over 7 years of expertise in designing, implementing

John holds a degree in Computer Science and certifications in several cybersecurity domains, including Certifi

Key Skills:
- Network Security Architecture
- Firewall Management and Configuration
- Intrusion Detection and Prevention Systems (IDS/IPS)
- Virtual Private Networks (VPNs)
- Security Audits and Risk Assessments
- Cybersecurity Incident Response
- Network Monitoring and Traffic Analysis
- Vulnerability Assessment and Penetration Testing
- Data Encryption and Secure Communications

Certifications:
- CISSP (Certified Information Systems Security Professional)
- CEH (Certified Ethical Hacker)
- CCNA (Cisco Certified Network Associate)
- CompTIA Security+

Education:
BSc in Computer Science, XYZ University, 2012-2016

Professional Experience:
- Network Security Engineer at ABC Corp (2016-Present)
- IT Security Specialist at DEF Solutions (2014-2016)

Languages:
- English (Fluent)
- French (Intermediate)
"""

# Now, test the model with the Network Security Engineer-focused resume
pred(myresume)
```

```
np.str_('Network Security Engineer')
```

```
myresume = """
Sarah Williams is a dedicated and skilled advocate with over 10 years of experience in providing legal repres
```